Lab: 5

Title: To implement Boundary Fill and Flood Fill using Turbo C++

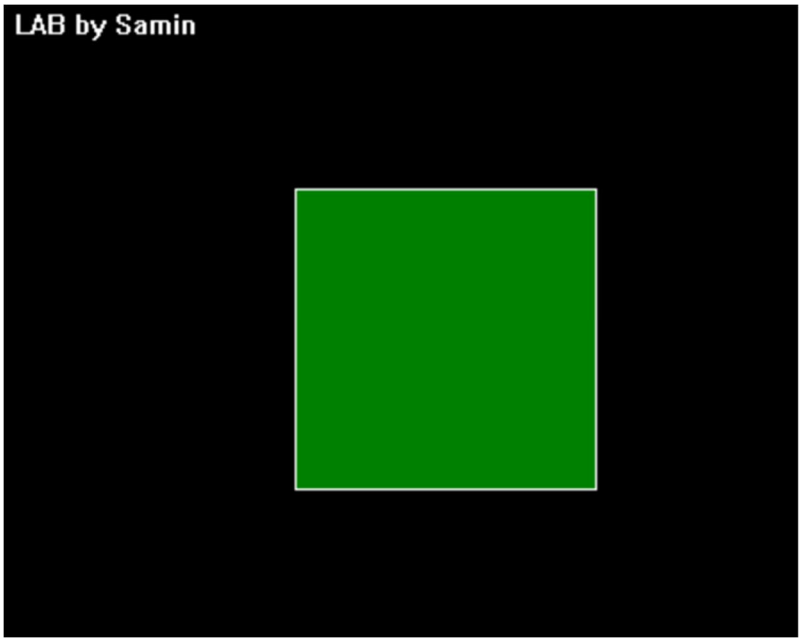
Program:

A. Boundary Fill:

```
#include <graphics.h>
#include <conio.h>
#include <stdio.h>
// Boundary Fill Function
void boundaryFill(int x, int y, int fill_color, int boundary_color)
{
int current = getpixel(x, y);
if (current != boundary_color && current != fill_color)
{
putpixel(x, y, fill_color);
boundaryFill(x + 1, y, fill_color, boundary_color);
boundaryFill(x - 1, y, fill_color, boundary_color);
boundaryFill(x, y + 1, fill_color, boundary_color);
boundaryFill(x, y - 1, fill_color, boundary_color);
}
}
int main()
{
int gd = DETECT, gm;
// Initialize graphics (Turbo C++ path)
initgraph(&gd, &gm, "C:\\TC\\BGI");// Display lab name
outtextxy(10, 10, "LAB by SAMIN");
// Draw a rectangle boundary
```

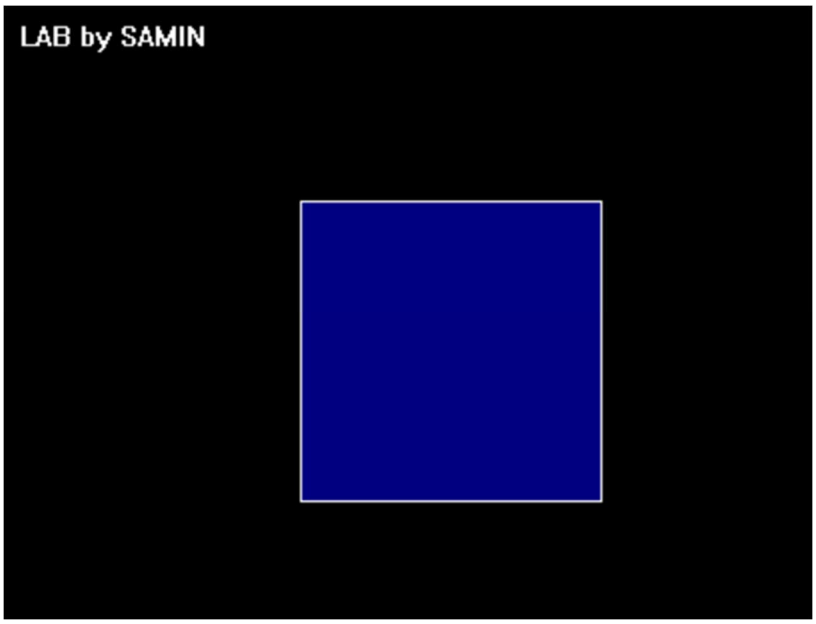
```
rectangle(150, 100, 300, 250);
// Fill inside the rectangle
boundaryFill(200, 150, GREEN, WHITE);
getch();
closegraph();
return 0;
}
```

Output:



B. Flood Fill

```
#include <graphics.h>
#include <conio.h>
#include <stdio.h>
// Flood Fill Function (4-connected)
void floodFill(int x, int y, int old_color, int new_color)
{
int current = getpixel(x, y);
if (current == old_color)
{
putpixel(x, y, new_color);
floodFill(x + 1, y, old_color, new_color);
floodFill(x - 1, y, old_color, new_color);
floodFill(x, y + 1, old_color, new_color);
floodFill(x, y - 1, old_color, new_color);
}
}
int main()
{
int gd = DETECT, gm;
// Initialize graphics for Turbo C++
initgraph(&gd, &gm, "C:\\TC\\BGI");
// Display Lab Name
outtextxy(10, 10, "LAB by SAMIN");// Draw a rectangle
rectangle(150, 100, 300, 250);
// Flood fill inside the rectangle
floodFill(200, 150, BLACK, BLUE);
```

```
getch();
closegraph();
return 0;
}
```

Output: